分布式操作系统综合习题集

目录

1 一、概念与体系结构 2

2 二、通信与同步 5

3 三、资源管理与调度 8

4 四、文件系统 12

5 五、计算分析题 14

1 一、概念与体系结构

1. **分布式系统的基本定义与区别：**

   (a) 什么是分布式操作系统？它的结构模型有哪些？层次如何划分？

   【参考答案】：

   定义：分布式操作系统是配置在分布式计算机系统上的操作系统。它负责管理整个系统的处理机、存储器、设备和文件等资源，向用户提供一个统一的、透明的接口，使用户把整个分布式系统看作一台单一的计算机（单一系统映像）。

   结构模型：

   - 小型机模型 (Minicomputer Model)
   - 工作站模型 (Workstation Model)
   - 处理机池模型 (Processor Pool Model)
   - 混合模型 (Hybrid Model)

   层次划分：一般可划分为：硬件层 → 操作系统内核层 → 分布式服务层（中间件）→ 应用层。

   (b) 请指出分布式系统与一般的计算机网络系统的本质区别是什么？

   【参考答案】：

   主要区别在于高层软件（操作系统）的特性：

   - **透明性（单一系统映像）：**分布式系统中，用户不知道也不需要知道资源位于哪个物理节点，系统自动管理；而网络操作系统中，用户通常需要显式指定远程机器（如 telnet, ftp）。
   - **资源管理：**分布式系统进行全局资源管理；网络系统由各节点独自管理本地资源。

   (c) 分布式系统与集中式计算机系统的主要区别是什么？

   【参考答案】：

   - **多处理机：**分布式系统拥有多个自主的处理机。
   - **无共享：**分布式系统各节点没有全局共享内存，也没有全局时钟（时钟不同步）。
   - **通信：**分布式系统必须通过消息传递进行通信。

2. **系统透明性**：什么是系统的透明性？分布式系统的透明性表现在哪几个方面？请解释其含义。为什么说它是分布式系统的一个主要设计目标？

【参考答案】:

定义：透明性是指系统对用户和应用程序隐藏其分布式特性的能力，使用户觉得像是在使用单机系统。

表现方面（ISO 参考模型）：

- **访问透明性**：隐藏数据表示和资源访问机制的差异。
- **位置透明性**：用户无需知道资源所在的物理位置。
- **并发透明性**：多个用户并发访问资源而不互相干扰。
- **复制透明性**：用户不知道资源有几个副本及其位置。
- **故障透明性**：掩盖故障，系统在部分故障下仍能继续工作。
- **迁移透明性**：资源在不同节点间移动不影响用户使用。
- **持久透明性**：隐藏资源在内存和磁盘间的移动。

原因：它是主要设计目标，因为极大地简化了应用程序的开发，提高了系统的易用性、可维护性和用户体验。

3. **客户/服务器（C/S）模式**：什么是客户/服务器模式？分布式系统为什么可以采用这一模式？请以实例说明之。

【参考答案】:

定义：将分布式系统中的进程分为两类：请求服务的进程（客户）和提供服务的进程（服务器）。它们通常运行在不同的机器上，通过消息传递进行通信。

原因：

- **模块化**：清晰划分功能，易于设计和实现。
- **灵活性与扩展性**：可以独立升级客户机或服务器，易于添加新服务。
- **资源分离**：适合分布式环境（如数据集中管理，界面分散处理）。

实例： Web 服务。浏览器（客户）发送 HTTP 请求，Web 服务器（如 Apache）接收请求、处理并返回 HTML 页面。

4. **分布式系统相对于集中式系统的优点和缺点**：说明分布式系统相对于集中式系统的优点和缺点。从长远的角度看，推动分布式系统发展的主要动力是什么？

【参考答案】:

优点：

- **经济性**：微处理机提供了比大型主机更好的性能价格比。
- **速度**：分布式系统总的计算能力比单个大型主机更强。
- **分布性**：具有固有的分布性，适用于空间上分散的机器。
- **可靠性**：具有极强的可靠性，如果一个节点崩溃，整个系统还可以继续运行。
- **可扩展性**：计算能力可以逐渐增加。

缺点：

- **软件问题**：目前分布式操作系统开发的软件太少。
- **通信网络问题**：网络的信息丢失或饱和会抵消建立分布式系统所获得的优势。
- **安全问题**：数据的易于共享也容易造成对保密数据的访问。

推动发展的主要动力：大量个人计算机的存在和人们共同工作于信息共享的需要，这种信息共享必须是以一种方便的形式进行，而不受地理或人员、数据以及机器的物理分布的影响。

5. **多处理机系统和多计算机系统的区别**：多处理机系统和多计算机系统有什么不同？

【参考答案】：

本质区别：多处理机系统是共享存储器的计算机系统，多计算机系统是不共享存储器的计算机系统。

详细说明：

- **多处理机系统**：所有 CPU 共享统一的虚拟地址空间，分为基于总线和基于交换两种。
  - 基于总线：多个 CPU 连接到一条公共总线和一个存储器模块。
  - 基于交换：存储器划分为多个模块，通过纵横式交换器连接。
- **多计算机系统**：每个计算机有它自己的存储器，分为基于总线和基于交换两种。
  - 基于总线：每个 CPU 与自身存储器直接相连，通过共享网络彼此相连。
  - 基于交换：处理器之间消息通过互联网进行路由。

6. **分布式操作系统的特点**：真正的分布式操作系统的主要特点是什么？

【参考答案】：

- 有一个单一的、全局的进程间通信机制。
- 进程管理必须处处相同。
- 文件系统相同。
- 使用相同的系统调用接口。

7. **微内核技术**：在分布式操作系统中，为什么采用微内核技术？通常微内核提供哪些服务？

【参考答案】:

采用微内核技术的原因：

- **高度模块化**：每一个服务都有定义好的接口，每个用户都可以访问任何服务，服务与位置独立。
- **高度灵活性**：具有添加、删除和修改服务的功能。
- **用户定制**：用户可以自定义服务。

微内核提供的服务：

- 进程间通信机制。
- 某些内存管理功能。
- 少量的底层进程管理和调度。
- 低层输入/输出服务。

2 二、通信与同步

1. **通信原语**：何谓分布式系统中的原语？请给出其消息传递中以下概念的描述：

- 阻塞（同步）与非阻塞（异步）原语
- 有缓冲与无缓冲原语
- 可靠与不可靠的原语

【参考答案】:

原语：指由若干条指令组成的、用于完成一定功能的一个过程，执行过程中不可分割。在通信中指最基本的发送/接收操作。

概念描述：

- **阻塞与非阻塞**：阻塞原语指调用后进程暂停执行，直到操作完成（如消息发出或收到）才返回；非阻塞原语指调用后立即返回，进程继续执行，不等待操作完成。
- **有缓冲与无缓冲**：有缓冲指系统内核提供缓冲区暂存消息，发送方拷贝数据到内核缓冲区即可；无缓冲指消息直接从发送方地址空间拷贝到接收方，通常需要收发双方”握手”同步。
- **可靠与不可靠**：可靠原语保证消息被对方正确接收（通常通过确认机制 ACK）；不可靠原语只负责发送，不保证送达（如 UDP）。

2. **远程过程调用（RPC）：**

   (a) 简述远程过程调用的步骤，并以实例说明。

   (b) 远程过程调用如何获得过程所在的节点（绑定）？有哪几种方法？它们各有什么优缺点？

   【参考答案】：

   (1) 步骤： 1. 客户调用客户存根 (Client Stub)；2. 存根打包参数 (Marshalling)，调用系统发送消息；3. 消息经网络传至服务器；4. 服务器内核将消息交给服务器存根 (Server Stub)；5. 存根解包参数，调用服务器过程；6. 执行并返回结果（逆向重复上述步骤）。**实例：**客户端调用 ‘read(fd, buf, n)‘，实际由 Stub 将参数打包发送给文件服务器，服务器执行实际读盘操作。

   (2) 绑定方法：

   - **集中式名字服务器（Name Server）：**服务器启动时注册，客户机向名字服务器查询。
     - 优点：简单，易于管理。缺点：存在单点故障，可能成为瓶颈。
   - **广播查找：**客户机广播请求，目标服务器响应。
     - 优点：灵活，无单点故障。缺点：增加网络负载。

3. **互斥与调度算法（Lamport）：**

   (a) 处理机调度要考虑哪些因素？

   (b) Lamport 算法完成一次临界区操作需要发送什么样的消息（几个消息）？

   (c) 该算法是否可以进行改进？若可以，请给出改进方法。

【参考答案】:

(1) 调度因素：系统吞吐量、响应时间、CPU 利用率、公平性、任务优先级、负载平衡。

(2) Lamport 消息数：需要 $3(N-1)$ 条消息。

- 请求 (Request)：向所有其他节点发送 $(N-1)$。
- 应答 (Reply)：所有其他节点回复 $(N-1)$。
- 释放 (Release)：向所有其他节点发送 $(N-1)$。

(3) 改进：可以改进，如 Ricart-Agrawala 算法。**方法**：将" 应答" 和" 互斥判断" 合并。节点收到请求后，若自己不在临界区且无更早请求，则立即回复；否则推迟回复。这样省去了释放消息（通过收到回复来隐含确认），仅需 $2(N-1)$ 条消息。

4. **服务器进程编址与定位**：客户为了发送消息给服务器，它必须知道服务器的地址。试给出服务器进程编址的几种方法，并说明如何定位进程。

【参考答案】:

编址方法：

- **机器号加进程号**：内核使用机器号将消息发送到适当的机器，用进程号决定接收进程。
- **随机地址加广播定位**：进程在大的地址空间中随机指定标识号，在支持广播的 LAN 中通过广播定位。
- **ASCII 码访问**：客户机向名字服务器发送请求，名字服务器将 ASCII 服务器名映射成服务器地址。

5. **C/S 模式消息可靠传输**：说明在 C/S 模式下解决消息可靠传输的三种方法。

【参考答案】:

- **重新定义非可靠的 send 语义**：系统无法保证消息发送成功，可靠通信依赖于用户处理。
- **内核确认机制**：接收机器的内核给发送机器的内核发送确认消息，发送内核收到确认后才释放用户进程。
- **阻塞客户加应答确认**：客户发送消息后阻塞，服务器内核将应答作为确认消息，客户进程阻塞到应答到来。

6. **RPC 崩溃处理**：在 RPC 调用时，如果服务器或客户机崩溃了，各有哪些解决方法？

【参考答案】:

服务器崩溃：

- **至少一次语义**：等待服务器重新启动并重发请求，保证 RPC 至少执行一次。
- **至多一次语义**：立即放弃并报告失效，确保 RPC 至多执行一次。
- **精确一次语义**：确保 RPC 恰好执行一次。

客户机崩溃：

- **根除**：在发送 RPC 前做日志，系统重启后检查日志并杀死孤儿进程。
- **再生**：将时间分成有序的纪元，客户重启时广播新纪元消息，结束所有远程计算。
- **温和再生**：服务器检查远程计算，只有找不到所有者的才终止。
- **过期**：每个 RPC 分配标准时间 T，超时则终止并重新分配。

3 三、资源管理与调度

1. **任务分配与资源共享：**

   (a) 如何理解分布式系统中的多资源共享和单资源共享？它们有哪些异同？

   (b) 简述任务分配与负载平衡的联系与区别。

   (c) 分布式系统中的动态任务分配应建立什么样的策略？需要考虑哪些因素？

【参考答案】:

(1) 理解：单资源共享通常指多个客户竞争单一服务器资源；多资源共享涉及跨多个节点的资源协同（如分布式数据库）。**(2) 任务分配 vs 负载平衡**：任务分配是指将进程指派给处理机的过程（通常指静态的或初始的）；负载平衡是任务分配的一种动态策略，目标是使各节点负载趋于平均，避免忙闲不均。**(3) 动态策略**：应建立包含以下四个部分的策略：1. **传输策略**（何时迁移）；2. **选择策略**（迁移哪个任务）；3. **位置策略**（迁移到哪）；4. **信息策略**（如何收集负载信息）。需考虑因素：CPU 负荷、通信开销、迁移代价。

2. **进程迁移**：在抢占式自适应算法中，迁移已经部分执行的任务的代价为什么比较大？你认为在哪几种情况下这种迁移会发生？

【参考答案】:

代价大原因：需要提取并传输进程的当前完整状态（上下文），包括虚拟内存映像（代码、数据、堆栈）、寄存器状态、打开的文件描述符、挂起的 I/O 操作等。数据量大且处理复杂。

发生情况：1. **负载极度不均**：当前节点过载严重，甚至影响服务质量。2. **硬件维护/故障**：节点即将停机。3. **移动计算**：用户物理位置移动。4. **资源亲和性**：进程需要访问大量位于远程节点的数据。

3. **私有工作空间实现事务**：举例说明用私有工作空间实现事务处理时的基本思想。

【参考答案】:

基本思想：在进程开始一个事务时给它分配一个包含所有需要访问文件的私有工作空间，在事务提交或终止前，所有读写操作都在私有空间而不是真正的文件系统中进行。

优化方法：

- **父辈工作区指针**：私有空间中只包含指向父辈工作区的指针，最顶层事务的工作区是真正的文件系统。
- **索引节点复制**：只复制索引节点（包含文件所在磁盘块位置的信息），不复制全部文件内容。

4. **两阶段提交协议**：说明在分布式系统中实现原子性提交的两阶段提交协议的基本思想及其优点。

【参考答案】:

基本思想：有一个进程作为协调者（执行事务的进程）。

第一阶段（准备提交）：

(a) 协调者向日志写入 Prepare。

(b) 向所有服务器发送准备提交消息。

(c) 服务器检查是否可以准备提交，若可以则向日志写入 Ready 并向协调者发送准备好消息。

第二阶段（提交/撤销）：

(a) 协调者接收所有响应后决定提交还是撤销。

(b) 协调者写入日志并发送决定消息。

(c) 服务器接到消息后写入日志并发送结束消息。

优点：确保所有服务器要么全部提交，要么全部撤销，保证原子性。

5. **集中式死锁检测的假死锁**：举例说明为什么使用集中式的死锁检测算法会产生假死锁，并给出一种解决办法。

【参考答案】:

假死锁原因：协调者收到的消息可能存在时序问题。例如：进程 B 先释放资源 R 再请求资源 T，但协调者先收到请求 T 的消息，导致检测到不存在的环路。

解决办法：使用 Lamport 算法提供全局统一的时间戳，对协调者收到的消息按时间戳排序后再处理。

6. **线程包实现方法**：叙述实现线程包的方法及其优缺点。

【参考答案】:

1. 在用户空间实现线程：是将线程包完全放到用户空间中去，内核对此一无所知。**优点**：在不支持线程的操作系统中实现；线程切换比使用内核陷阱快一个数量级；允许每个进程有自己定制的调度算法。**缺点**：阻塞调用难以实现；需要将系统调用改为非阻塞（如使用 SELECT）；需要使用旋转锁和信号中断来实现调度。

2. 在内核中实现线程：当线程创建或撤销时发出内核调用，由内核完成工作。内核中每个进程都有一张包含线程信息的表。**优点**：容易实现调度。**缺点**：系统开销大。

3. 调度者行为：内核分配一定数量的虚拟处理机给每个进程，让运行期系统将线程分配给处理机。**优点**：效率高，很好地解决了控制权由阻塞线程传递给非阻塞线程的问题。**缺点**：可能产生死锁，与分层系统的内在结构违背。

7. **分布式启发算法**：说明发送者发起的分布式启发算法和接收者发起的分布式启发算法及各自的主要缺点。

【参考答案】:

发送者发起的算法：由发送者（负载高的节点）主动发起迁移请求，寻找合适的接收者。**主要缺点**：发送者可能频繁发起请求，增加网络开销；可能导致负载振荡。

接收者发起的算法：由接收者（负载低的节点）主动请求任务。**主要缺点**：负载低的节点可能不需要任务，造成资源浪费；响应时间可能较长。

8. **Byzantine 将军问题**：举例说明 Lamport 等人提出的算法是如何解决 Byzantine 将军问题的。

【参考答案】:

问题描述： n 个将军通过信使通信，其中可能有 m 个叛徒，需要达成一致的进攻或撤退命令。

算法思想：

- 每个将军收集所有其他将军的命令信息。
- 使用递归算法交换和验证命令。
- 如果收到超过 $n-m$ 条一致的消息，则接受该命令。

结论： 当 $n > 3m$ 时，可以达成正确的一致性。

9. **三模冗余：** 简述三模冗余的基本思想，并举例说明三模冗余能否处理 Byzantine 故障。

【参考答案】:

基本思想： 使用三个相同的模块执行相同的任务，通过投票器选择多数一致的结果作为输出。

处理 Byzantine 故障：

- **可以处理：** 随机故障（如模块产生错误输出）。
- **不能处理：** Byzantine 故障（模块故意产生不一致的错误输出）。

原因： 三个模块可能产生三种不同的输出，投票器无法判断哪个是正确的。

10. **处理机分配算法：** 举例说明采用图论确定性算法进行处理机分配的实现方法。

【参考答案】:

基本思想： 将处理机分配问题建模为图论问题。

- **节点：** 表示处理机或进程。
- **边：** 表示处理机与进程之间的兼容性或分配关系。
- **目标：** 在满足约束条件的情况下找到最优或可行的分配方案。

方法： 使用匹配算法（如匈牙利算法）或搜索算法找到满足所有进程需求的处理机分配。

4 四、文件系统

1. **文件系统设计**：你认为文件系统应该分成为哪几个模块？按照 OOD（面向对象设计）的思想，应该设计几个对象？

   【参考答案】:

   模块划分： 1. **目录服务模块：**管理文件名到 ID 的映射。2. **文件服务模块：**管理文件属性和数据读写。3. **块服务/扁平文件模块：**管理磁盘块和物理存储。4. **客户接口模块：**提供 API。

   OOD 对象： 1. **文件对象 (File)** 2. **目录对象 (Directory)** 3. **文件系统对象 (FileSystem)** 4. **事务对象 (Transaction)**

2. **命名与定位**：如何给一个文件命名多个名字？文件组是如何被定位的？

   【参考答案】:

   多重命名：

   - **硬链接 (Hard Link)**：多个目录项指向同一个索引节点 (I-node)。
   - **符号链接 (Symbolic Link)**：建立一个特殊文件，其内容是另一个文件的路径名。

   文件组定位：通常通过位置数据库（映射表）查找，或使用哈希算法映射，或在局域网内广播查询。

3. **文件传输模式**：使用上载/下载模式的文件服务器系统与使用远程访问模式的文件系统之间有什么区别？

   【参考答案】:

   上载/下载模式：文件完整地从服务器传输到客户（下载）或从客户传输到服务器（上载）。客户得到文件的本地副本，所有操作在本地副本上进行，操作完成后需要将修改后的文件上传回服务器。**优点：**简单，一次性传输后操作速度快。**缺点：**不适合频繁访问大型文件，网络开销大。

   远程访问模式：文件保留在服务器上，客户通过远程过程调用直接访问文件的各个部分。**优点：**适合频繁访问少量数据的情况，节省带宽。**缺点：**每次访问都需要网络通信，延迟较高。

4. **文件共享语义**：分布式系统中处理共享文件的四种方法（文件共享的四种语义）。

   【参考答案】:

- **UNIX 语义**：每次 read 看到最后一次 write 的结果。
- **会话语义**：打开文件后看到的是文件的私有视图，关闭时才写回。
- **不可变语义**：文件一旦创建就不可修改，任何读操作都看到相同的版本。
- **事务语义**：使用事务机制保证文件操作的一致性。

5. **无状态与有状态服务器**：说明无状态服务器和有状态服务器，并说明二者的区别。

【参考答案】:

无状态服务器：不保存客户的状态信息。每次请求都是独立的，服务器不记录客户之前访问过哪些文件或做了哪些操作。**优点**：简单、可靠；服务器崩溃后容易恢复；不限制同时访问的客户数。**缺点**：每个请求都需要携带完整的信息，开销较大。

有状态服务器：跟踪客户的状态信息（如打开的文件列表、当前位置等）。**优点**：可以优化后续操作，减少数据传输。**缺点**：服务器崩溃后状态丢失，恢复困难；需要管理客户状态，资源消耗大。

6. **客户高速缓存一致性**：说明保持客户高速缓存一致性的四种算法。

【参考答案】:

- **轮询（Poll）**：客户定期向服务器询问文件是否已修改。
- **租约（Lease）**：服务器授予客户一个时间段内的缓存有效性许可。
- **通知（Invalidation）**：服务器在文件修改时主动通知所有缓存该文件的客户。
- **访问控制**：客户每次访问都需与服务器确认。

7. **文件复制服务**：说明分布式系统提供文件复制服务的原因以及实现复制的三种方法。

【参考答案】:

原因：提高可用性、可靠性和性能（负载均衡）。

实现方法：

- **主拷贝算法**：有一个主副本，所有修改都先应用到主副本，再传播到从副本。
- **Gifford 算法**：使用读写定额机制，读取时从多个副本获取数据并验证一致性。**主动复制**：所有副本都可以接收修改请求，通过协议保证一致性。

8. **更新复制文件算法**：说明更新复制文件的两种主要算法：主拷贝算法、Gifford 算法。

【参考答案】:

主拷贝算法：

- 有一个主副本（Primary Copy）负责协调所有更新。
- 客户的所有写请求都发送到主副本。
- 主副本按顺序应用更新，然后传播到所有从副本。
- 读取操作可以从任何副本进行。

Gifford 算法：

- 定义读定额 N_r 和写定额 N_w。
- 读操作必须从至少 N_r 个副本读取并比较结果。
- 写操作必须更新至少 N_w 个副本。
- 满足 $N_r + N_w > N$ 且 $N_w > N/2$ 以保证一致性。

5 五、计算分析题

1. **多线程性能计算（场景 A）**：考虑单/多线程文件服务器读一个文件的情况。如所需的数据在高速缓存中，那么接受一个请求、分配它、做必要处理总共花 15ms。若在 1/3 的时间的情况下，需要一次磁盘操作，还需多花 **30ms**，在这段时间内线程睡眠。试问：

   (1) 若使用单线程，每秒可处理多少个请求？

   (2) 若使用 2 个线程，每秒可处理多少个请求？（注意：写出计算过程）

【参考答案】:

(1) 单线程：平均处理时间 $T_{avg} = \frac{2}{3} \times 15\text{ms} + \frac{1}{3} \times (15 + 30)\text{ms} = 10 + 15 = 25\text{ms}$。每秒处理请求数 $= 1000/25 = 40$ 次。

(2) 双线程：

- 每个请求占用 CPU 时间：15ms。
- 每个请求平均占用磁盘时间：$30 \times \frac{1}{3} = 10\text{ms}$。
- CPU 是瓶颈（$15 > 10$）。双线程可以利用磁盘等待时间处理计算。
- 理论最大吞吐量由 CPU 决定：$1000/15 \approx 66.67$ 次。

- 两个线程足以在大部分时间重叠计算与 I/O，因此结果接近理论极限 **66.67 次**。

2. **多线程性能计算（场景 B）**：考虑单/多线程文件服务器读一个文件的情况。如所需的数据在高速缓存中，那么接受一个请求、分配它、做必要处理总共花 15ms。若在 1/3 的时间的情况下，需要一次磁盘操作，还需多花 **75ms**，在这段时间内线程睡眠。试问：

   (1) 若使用单线程，每秒可处理多少个请求？

   (2) 若使用 2 个线程，每秒可处理多少个请求？（注意：写出计算过程）

   【参考答案】:

   (1) 单线程：平均时间 $T_{avg} = \frac{2}{3} \times 15 + \frac{1}{3} \times (15 + 75) = 10 + 30 = 40\text{ms}$。每秒处理请求数 $= 1000/40 = 25$ 次。

   (2) 双线程：

   - CPU 需求：15ms/req。
   - 磁盘需求平均值：$75 \times \frac{1}{3} = 25\text{ms/req}$。
   - 此时 **磁盘 I/O 是瓶颈** $(25 > 15)$。
   - 即使线程数无限，系统吞吐量也受限于磁盘子系统：$1000/25 = 40$ 次。
   - 对于双线程，由于 75ms 等待时间远大于 15ms 处理时间，两个线程虽然能提升并发，但会受限于磁盘速度。理论上限为 **40 次**。

3. **ATM 系统中断处理**：一个 ATM 系统以 OC-3 的速率传递信元，每个包 48 字节长，刚好放进一个信元，一个中断耗时 1 s。试问：

   (1) CPU 用于中断处理的时间是多少？

   (2) 如果包长是 1024 字节，CPU 用于中断处理的时间是多少？

   【参考答案】:

   OC-3 的数据传输速率为 155.520Mbps。**(1)** 信元总长 53 字节（48 数据 +5 头）。每秒传递包（信元）的数目为（按网络常用 $1M = 10^6$ 计算，若按存储 1024^2 则略有不同，此处沿用题目常见解法逻辑）：

$$\frac{155.52 \times 10^6}{8 \times 53} \approx 366792 \quad (\text{若按}1024^2\text{算则为}384609)$$

   此处采用 384609（题目原解法数据）。CPU 用于中断处理的时间是：

$$1 \times 10^{-6} \times 384609 \approx 0.38\text{s}$$

(2) 如果包长是 1024 字节，一个包拆分为 $\lceil 1024/48 \rceil = 22$ 个信元。但题目意指"每个包产生一次中断"（重组后）。每秒传递包的数目为：

$$\frac{384609}{22} \approx 17482$$

CPU 用于中断处理的时间是：

$$1 \times 10^{-6} \times 17482 \approx 0.017\text{s}$$

4. **远程过程调用（RPC）时间计算**：假设一个空 RPC（0 字节数据）需时间 1.0ms，每增加 1k 数据，时间增加 1.5ms。如果要从文件服务器读 32k 的数据，试计算：

   (1) 一次读取 32k 数据的 RPC 所需的时间

   (2) 32 次读取 1k 数据的 RPC 所需的时间

   【参考答案】:

   (1) 一次读取 32k 数据的 RPC 所需时间：

   $$T = 1.0 + 32 \times 1.5 = 1.0 + 48 = 49\text{ms}$$

   (2) 32 次读取 1k 数据所需时间：每次调用时间 $= 1.0 + 1 \times 1.5 = 2.5\text{ms}$。总时间 $T_1 = 32 \times 2.5 = 80\text{ms}$。

5. **时钟同步**：考虑一个分布式系统中的两台机器。这两台机器的时钟假设都每毫秒滴答 1000 次，但实际上只有一个是这样，而另一个一毫秒仅滴答 990 次。如果 UTC 每分钟更新一次，那么时钟的最大偏移量将是多少？

   【参考答案】:

   正常时钟每秒滴答 $1000 \times 1000 = 10^6$ 次。慢时钟每秒滴答 $990 \times 1000 = 9.9 \times 10^5$ 次。两者每秒相差 $10,000$ 次滴答，相当于 10 毫秒。一分钟（60 秒）后，误差积累为：

   $$60 \times 10\text{ms} = 600\text{ms}$$

6. **文件服务器性能分析**：如果要求的数据在缓存中，收到任务请求、分配这个工作并做相应处理需要 15 毫秒；如果要进行磁盘读写（占用 1/3 的时间），则需要多使用 75 毫秒，在磁盘操作时线程休眠。请分别计算：

(1) 单线程文件服务器每秒能处理多少条请求？

(2) 多线程文件服务器每秒能处理多少条请求？

【参考答案】:

(1) **单线程：**

$$T = \frac{2}{3} \times 15 + \frac{1}{3} \times (15 + 75) = 10 + 30 = 40\text{ms}$$

吞吐量：$1000/40 = 25$ 次/秒。

(2) **多线程：**CPU 处理时间 $= 15$ms。磁盘平均处理时间 $= 75/3 = 25$ms。系统瓶颈在于磁盘 $(25 > 15)$。最大处理能力 $= 1000/25 = 40$ 次/秒。*注：若假设有无限磁盘带宽或题目仅考察 CPU 处理能力上限，则结果为 $1000/15 \approx 66.67$。但根据题目条件瓶颈分析，应为 40。*

7. **投票表决算法：**一个文件在 10 个服务器上复制，试列举投票表决算法所允许的读数定额与写数定额的所有组合。

【参考答案】:

根据 Gifford 投票算法：1. $N_r + N_w > N$ (读写互斥) 2. $N_w > N/2$ (写写互斥)

已知 $N = 10$，则必须 $N_w > 5$。满足条件的组合如下：

```
Nw=6, Nr=5,6,7,8,9,10
Nw=7, Nr=4,5,6,7,8,9,10
Nw=8, Nr=3,4,5,6,7,8,9,10
Nw=9, Nr=2,3,4,5,6,7,8,9,10
Nw=10, Nr=1,2,3,4,5,6,7,8,9,10
```

8. **文件备份：**文件服务器的一个主存来连续地存储文件，当文件增大超出了当前所在单元，该文件要进行备份。假设一般文件长为 20M 字节，每拷贝 32 字节的字需要 200 纳秒（10^{-9} 秒），那么：

(1) 一秒钟能拷贝多少文件？

(2) 你能给出一种方法，备份文件而又不一直占用文件服务器 CPU 的时间？

【参考答案】:

(1) 拷贝一个文件所需时间：

$$T = \frac{20 \times 10^6}{32} \times 200 \times 10^{-9} = 625,000 \times 2 \times 10^{-7} = 0.125\text{s}$$

一秒钟能拷贝的文件数：

$$\frac{1}{0.125} = 8 \text{ 个文件}$$

(2) **方法：使用 DMA (直接存储器访问)** 技术。CPU 只需设置源地址、目的地址和字节数，然后启动 DMA 控制器。数据传输由 DMA 硬件完成，CPU 可以并行处理其他任务，传输完成后通过中断通知 CPU。

9. **Dash 交换式多处理机系统：**说明在 Dash 交换式多处理机系统中，如何读写一个存储器中的字。

【参考答案】:

读操作：

1. 处理器发起读请求，包含目标存储器地址。
2. 请求通过互连网络路由到相应的存储器模块。
3. 存储器模块返回数据到请求处理器。

写操作：

1. 处理器发起写请求，包含目标地址和数据。
2. 请求路由到目标存储器模块。
3. 存储器模块更新数据并可能发送无效化或更新消息到其他缓存。

10. **多处理机系统写协议：**说明基于总线的多处理机系统中的 write-through 和 write-once 协议。

【参考答案】:

write-through（写直达）：

- 每次写操作都同时更新主存。
- 简单但总线流量大。

write-once（写一次）：

- 第一次写时同时更新主存并标记缓存行。
- 后续写只更新缓存。

- 需要监听总线来维护缓存一致性。

11. **分布式共享存储器模型**：简述分布式共享存储器的三种模型：严格一致性、顺序一致性、因果一致性，并举例说明。

【参考答案】:

严格一致性：所有读操作看到的是最近一次写入的值。实际难以实现。

顺序一致性：所有处理器看到的所有操作的执行顺序相同，但不要求与实际时间顺序一致。

因果一致性：只有因果相关的操作必须按顺序执行，不相关的操作可以并发执行。

举例：

- 进程 P1 写入 X，进程 P2 读取 X 后写入 Y，这两个操作有因果关系，必须按此顺序执行。
- 两个独立的进程分别写入不同的变量，顺序可以互换。

12. **全局事件定序**：在分布式系统中，为什么要有一个全局的事件定序机构？用什么算法可以实现的？举例说明。

【参考答案】:

原因：分布式系统中没有全局时钟，需要一种机制来确定事件的全局顺序，用于调试、日志、分布式算法等。

算法： Lamport 逻辑时钟算法。

实现方法：

- 每个进程维护一个逻辑计数器。
- 发送消息时，计数器加 1 并附加在消息中。
- 接收消息时，将本地计数器更新为 $\max(\text{本地值}, \text{消息时间戳}) + 1$。

举例：可以用于实现分布式互斥算法中的请求排序。

13. **Lamport 算法消息数与改进**： Lamport 算法在完成一次临界区操作需要发送几个什么样的消息？是否可以改进呢？

【参考答案】:

Lamport 算法消息数：需要 $3(N-1)$ 条消息。

- 请求 (Request)：向所有其他节点发送 $(N-1)$。

- 应答 (Reply)：所有其他节点回复 $(N-1)$。
- 释放 (Release)：向所有其他节点发送 $(N-1)$。

改进方法： Ricart-Agrawala 算法。**改进：**将" 应答" 和" 互斥判断" 合并。节点收到请求后，若自己不在临界区且无更早请求，则立即回复；否则推迟回复。这样省去了释放消息（通过收到回复来隐含确认），仅需 $2(N-1)$ 条消息。

14. **文件系统 OOD 设计：**文件系统中按照 OOD（面向对象设计）的思想，应该设计几个对象？这些模块、对象各实现什么样的功能？

【参考答案】:

OOD 对象设计：

- **文件对象 (File)：**管理文件属性和数据读写操作。
- **目录对象 (Directory)：**管理文件名到文件对象的映射。
- **文件系统对象 (FileSystem)：**管理整个文件系统的全局状态。
- **事务对象 (Transaction)：**提供事务操作，保证操作的原子性。

模块划分：

- 目录服务模块。
- 文件服务模块。
- 块服务/扁平文件模块。
- 客户接口模块。

15. **文件组定位：**文件组是如何被定位的？

【参考答案】:

方法：

- **位置数据库：**使用映射表记录文件组的位置信息。
- **哈希算法：**通过哈希函数将文件名映射到存储位置。
- **广播查询：**在局域网内广播查询请求。
- **分布式哈希表：**使用 DHT 等分布式索引机制。